

Detailed Technical Overview: MyDelivery, MyDelivery Admin, and B2C

Audience: Freshers with limited exposure to legacy Java EE / Spring stacks. Focus: Explain architecture, request flows, components, legacy technologies, and upgrade considerations.

1. MyDelivery (Customer Redelivery Portal)

1.1 Purpose

Interactive portal enabling end customers to reschedule delivery or choose alternative handling options for a shipment (consignment): alternative address, leave with neighbour, leave in absence, self collection, original address retry.

1.2 Architectural Layers

1. Presentation (MyDeliveryPresentation): Spring WebFlow + Tiles + Freemarker/JSP for step-by-step wizard.
2. WebService (MyDeliveryWebService): Spring MVC endpoints for integration/mobile clients (REST-like RPC).
3. Service Implementation (MyDeliveryServiceImpl): Core business logic, option computation, date availability, confirmation & printing.
4. Service API (MyDeliveryServiceApi): Interfaces / models / enums / validators used across modules.
5. External Integration (ACL classes): Adapters to external systems (APC, OSC, CommonCodes, StreamServe).
6. Reporting: StreamServe & JasperReports integration for PDF forms (PTL) and confirmation documents.

1.3 Key Legacy Technologies

- Java pre-17 (likely 6/7/8 features only).
- Spring WebFlow XML definitions (state machine in `redelivery-flow.xml`).
- Apache Tiles for page composition; Freemarker & JSP for view templates.
- OVal for validation (pre-Jakarta Validation alternative).
- AspectJ for caching (runtime weaving) rather than Spring Cache abstraction.
- SimpleDateFormat / Date (not thread-safe & outdated vs java.time).

1.4 WebFlow Entry

`index.jsp` forwards to `/flow/redelivery.html` (logical flow id) mapped by `webflow-servlet.xml` to `redelivery-flow.xml`. Start state `init` executes decorators to derive country, consignment visuals, and skip flags.

1.5 Core Domain Concepts

- Consignment: Shipment entity whose status and attributes drive available options.
- RedeliveryOption: Enum of possible customer choices.
- RedeliveryRequestBean: Mutable form backing bean for user-entered data before confirmation.
- RedeliveryRequest: Immutable validated request ready for persistence / email / printing.
- AvailableOptionsModel / DatesModel: Computed sets of permissible options and selectable date slots.
- ViolationsModel: Aggregated validation errors/warnings.

1.6 Request Flow (Web UI)

1. Init → Determine if user enters details or starts at options.
2. Enter Details → Validate consignment id & base inputs.
3. Fetch Options → Service collects consignment + country metadata; rules evaluate permitted RedeliveryOption set.
4. Options View → User chooses option (branching state machine).
5. Option Specific Page → Fetch available dates (logic differs by option: depot windows, address constraints, leave-in-absence rules).
6. Enter Contact Details → Collect phone(s), email, instructions, neighbor or alternative address lines.
7. Confirm → Factory builds RedeliveryRequest; validation ensures date validity, option eligibility, field lengths, character sets.
8. Print (optional) → Servlet triggers JasperReports/StreamServe to render PDF.
9. End → Redirect to external site.

1.7 Validation Workflow

- Field annotation (OVal) + manual helper (ValidationHelper) compile rule violations.
- FieldLengthConstants centralize min/max; errors mapped to user messages.
- Specialized rules: postcode supply rules by country, phone prefix validity, email format, option-specific mandatory fields.

1.8 External Integrations

- APC ACL: Consignment & address details.
- OSC / CommonCodes ACL: Country-level metadata (rules, feature toggles).
- StreamServe ACL: XML request/response for document generation.

1.9 Reporting Subsystem

Builders assemble printable data objects; labels provider resolves localized keys; printer merges data into Jasper template or triggers StreamServe; output streamed by servlet (application/pdf).

1.10 Caching

AspectJ `Caching` aspect wraps read-only lookups (e.g., country metadata). Modern alternative: Spring Cache with Caffeine/Redis.

1.11 Upgrade Considerations (toward Java 17 / Spring Boot)

- Replace WebFlow wizard with REST endpoints + SPA (React/Vue) maintaining client state.
 - Swap Oval for Jakarta Validation annotations.
 - Replace AspectJ caching with Spring Cache.
 - Convert POJOs to records where immutable (`ConfirmationModel` , `DepotAddressModel`).
 - Migrate legacy date handling to `java.time` (`LocalDate`, `ZonedDateTime`).
 - Consolidate XML config to JavaConfig / annotations.
-

2. MyDelivery Admin (Operational Back-Office)

2.1 Purpose

Internal application for operations staff to configure and oversee redelivery logic: enabling/disabling options per country, managing depot data, viewing planned deliveries, and printing internal reports.

2.2 Architectural Layers

- Presentation (MyDeliveryAdminPresentation): Spring MVC controllers (`DispatcherServlet mydeliveryadmin`), legacy JSP/Tiles views, security filters.
- Application Layer / Service Modules: Business services for configuration & operational data retrieval.
- CrossDependencies: Shared artifacts reused by admin and customer apps.
- EAR packaging: Aggregates admin modules for deployment (WebSphere/Tomcat).

2.3 Key Technologies

- Java EE 2.4 style web.xml (explicit servlet/filter declarations).
- Spring MVC (no WebFlow here, primarily controllers).
- Spring Security via DelegatingFilterProxy (`springSecurityFilterChain`) and custom `SpringSecurityHolderSessionStrategy` .
- Compression filter for HTTP responses.

2.4 Entry Points

`web.xml` maps all URLs (`/*`) to `DispatcherServlet ( mydeliveryadmin )`. The welcome-file list includes standard index pages (`index.html / .jsp`). Controllers (not displayed here) handle admin actions.

2.5 Functional Operations (Typical)

- Country Rule Management: Toggle option availability, update postcode rules.
- Depot Management: Create/update depot address, opening hours, capacity windows.
- Planned Delivery Views: Retrieve and print planned delivery details (`PrintPlannedDeliveryDetailsServlet`).

- Audit/Monitoring: Inspect recent redelivery requests & statuses.
- Security Context Setup: Custom filter ensures security context available early.

2.6 Security

- Filter chain applies authentication/authorization; session strategy manages principal storage.
- Legacy Log4j initialization via listener; modern setups would use Logback and central configuration.

2.7 Printing in Admin

Dedicated servlet (`PrintPlannedDeliveryDetailsServlet`) similar pattern to customer printing: gather data, assemble printable model, stream PDF.

2.8 Data Flow Example (Depot Update)

1. User loads depot edit form.
2. Controller fetches existing depot model via service (calls ACL for persistent store if needed).
3. User submits changes; validation checks field lengths, required address components.
4. Service persists updates; caches invalidated.
5. Response returns success message; optional audit record stored.

2.9 Upgrade Considerations

- Replace XML config with Spring Boot auto-configuration.
- Implement role-based access via method security annotations (`@PreAuthorize`).
- Introduce REST endpoints for CRUD; front-end can be SPA or maintain server-rendered pages.
- Unified error handling with `@ControllerAdvice` .
- Migration to Java 17 records for read-only admin DTOs.
- Observability: metrics for config changes, security events.

2.10 Differences From Customer App

- Focus on CRUD & management vs wizard flow.
- Heavy use of security filters.
- Print functions oriented to operational planning rather than consumer confirmation.

3. B2C (Notification & Alerting Service)

3.1 Purpose

Automated batch system sending time-sensitive notifications (email/SMS) to consumers based on shipment status changes & alert windows.

3.2 Architectural Components

- Spring Batch jobs defined via XML (`process-alerts-job.xml` , `process-consignment-statuses-job.xml`).
- Partitioning for parallel processing of time windows (grid-size=10).
- Readers: WindowReader executes parameterized SQL retrieving rows in a time window.
- Processors: AlertProcessor / ConsignmentStatusProcessor apply business rules & mappings.
- Writers: AlertUpdater / ConsignmentStatusAlertWriter persist updated alert statuses / generated alerts.
- Listeners: Job & Step listeners capture metrics and logging (B2CJobExecutionListener, B2CStepExecutionListener).
- Model Classes: Represent consignment status events, alerts, templates, notification types.
- Communication Layer: Alserter classes (ConsumerAlserter, ConsignmentStatusAlertAlserter) orchestrate channel dispatch.
- Properties: Externalized mappings (status2TemplateMapping, country2LanguageMapping, time zones, due date formats) loaded via PropertiesFactoryBean.

3.3 Flow: Consignment Status Job

1. Job starts (`processConsignmentStatusesJob`).
2. Partitioner splits processing into minute-based windows (ConsignmentStatusesPartitioner).
3. For each partition: WindowReader fetches statuses for that window.
4. ConsignmentStatusProcessor maps statuses to templates & alert types using properties (config-driven logic).
5. Writer persists resulting alerts (ConsignmentStatusAlertWriter).
6. Alserter may dispatch messages (if immediate send configured) – else subsequent alert job handles send.
7. Listeners log execution summary, custom exit codes (B2CExitStatus mapped by B2CExitCodeMapper).

3.4 Flow: Alerts Job

1. Job starts (`processAlertsJob`).
2. AlertsPartitioner defines alert windows.
3. WindowReader fetches alerts needing processing.
4. AlertProcessor applies mode-specific logic (e.g., RESEND vs INITIAL send).
5. ConsumerAlserter builds localized messages (country2LanguageMapping, templateNr2TypeMapping). Date/time formatting via DateTimeHelper.
6. AlertUpdater marks alerts as sent / updates statuses.
7. Job & Step listeners record performance and results.

3.5 Template & Localization Handling

- Properties map template numbers to types, statuses to templates, and countries to language/time zone for message personalization.
- Due date formats adapt output formatting per locale.

3.6 Error Handling

- `TemplateException` / `CommunicationException` wrap lower-level failures.
- Persistence exceptions handled by Spring Batch transaction rollback (chunk processing with commit-interval).

3.7 Upgrade Considerations

- Move XML batch config to Java DSL (Spring Batch 5) under Java 17.
- Replace `PropertiesFactoryBean` with type-safe configuration (records + YAML).
- Introduce retry/circuit breaker via `Resilience4j` for external communications.
- Use `java.time` exclusively (remove custom `DateTimeHelper` reliance on legacy Date APIs).
- Add structured logging & metrics (Micrometer, OpenTelemetry).
- Containerize jobs; orchestrate via Kubernetes CronJobs.

3.8 Scaling & Performance

- Partitioning increases parallelism; grid-size tune based on DB throughput.
- Commit-interval controls transaction size; adjust to balance memory & commit overhead.

3.9 Key Model Classes (Simplified Roles)

- `ConsignmentStatus`: Represents an event (timestamp, status code).
- `ConsignmentStatusAlert`: Alert pending or dispatched referencing a consignment status.
- `NotificationType` / `AlertType` / `TemplateType`: Enums guiding template selection & channel logic.

3.10 Fresher Debug Path

1. Run a job with small test window (override job parameters).
 2. Set breakpoint in `Processor` to inspect mapping logic & loaded property sets.
 3. Inspect `Writer` to confirm DB persistence of alert changes.
-

4. Shared Components

4.1 Purpose

Centralize domain models, async processing utilities, shared services (`service-api/service-impl`), and location services across applications.

4.2 Typical Packages

- `entity-api` / `entity-impl`: Core domain entities (`Consignment`, `Depot`, `Location`) + implementations.
- `service-api` / `service-impl`: Shared service contracts (e.g., location lookup, domain enquiries).
- `delivery-domain-properties`: Common property container types.

- delivery-async-process: Generic asynchronous task helpers.
- delivery-domain-enquiry: APIs for complex read queries (aggregated reporting).
- MDLocationService: Specialized geocoding & depot resolution logic.

4.3 Integration Pattern

Applications depend on API modules; implementations wired via Spring contexts (XML currently). Migration path: publish as Spring Boot starters or internal Gradle/Maven modules with JavaConfig.

4.4 Modernization

- Convert entity POJOs to records if immutable.
- Introduce packaging rules: "api" modules should not depend on implementation modules.
- Add contract tests to ensure API backward compatibility.

5. Support Folders

Contain business & technical documentation: requirements specs, architecture diagrams, test plans, deployment guides. Use these to understand the "why" behind code decisions before refactoring.

6. Cross-Cutting Technicalities Explained for Freshers

Concept	Old Approach	Modern Approach	Why Modern
WebFlow State Machine	XML definitions manage page transitions	REST + client-side router / state mgmt (React/Vue)	Easier to evolve & test
Validation (OVal)	Custom library + helper classes	Jakarta Validation (Bean Validation)	Standard, ecosystem support
Date Handling	java.util.Date + SimpleDateFormat	java.time (LocalDate, ZonedDateTime)	Thread-safe, clearer semantics
Caching	AspectJ weaving	Spring Cache (Caffeine/Redis)	Simpler configuration, metrics
Configuration	Many XML + .properties	application.yml + JavaConfig	Reduced verbosity, better environment profiles

Concept	Old Approach	Modern Approach	Why Modern
Reporting	JasperReports + StreamServe XML	Service-side PDF (OpenPDF/FlyingSaucer) or client render	Flexibility, fewer proprietary dependencies
Batch (XML)	Spring Batch XML config	Spring Batch Java DSL + Cloud scheduling	Maintainability & observability

7. Suggested Learning Path

1. Understand layered architecture & domain vocabulary.
2. Trace one MyDelivery UI flow end-to-end (consignment → confirmation).
3. Explore Admin printing & rule configuration (difference from consumer flow).
4. Run a B2C batch job in test, observe partitioning & processing.
5. Review shared components (entity-api) to identify reuse points.
6. Read modernization roadmap to align future changes.

8. Migration High-Level Checklist (All Apps)

- Set Java version to 17.
- Replace XML configs with JavaConfig / annotations.
- Introduce unified logging & tracing.
- Migrate validation & date/time handling.
- Introduce containerization (Docker) & CI pipelines.
- Implement security hardening (JWT for APIs, method-level auth for Admin, sanitized inputs for UI).

9. Common Pitfalls & Tips

- Avoid modifying ACL integrations without understanding external API contracts.
- Do not introduce business logic into view templates – keep logic in services.
- Ensure thread-safety when adding shared caches or formatters (prefer dependency injection of immutable services).
- When refactoring, keep backward-compatible interfaces until clients migrated.

10. Glossary (Extended)

- Partitioning (Batch): Splitting workload into parallel segments.
- Chunk Processing: Reading N items, processing them, then writing in a single transaction.
- PropertiesFactoryBean: Legacy Spring bean for loading .properties into a map.
- DispatcherServlet: Core Spring MVC front controller handling HTTP requests.

- Anti-Corruption Layer (ACL): Pattern to isolate internal domain from external service peculiarities.
 - Decorator (UI Flow): Prepares model & view context for a specific flow step.
-

11. Next Steps for Freshers

- Pair program a small feature (e.g., add a new validation constraint) using legacy approach, then prototype modern replacement.
- Write an integration test for an existing B2C processor mapping property-driven templates.
- Document findings & share with team for review.

Welcome to the platform engineering journey!